

Contents

Chapitre 3 — Prototype Minimal de Chatbot Embarqué	1
1. Introduction et objectifs	2
2. Architecture du prototype	2
2.1 Vue d’ensemble	2
2.2 Pile technologique	2
2.3 Pipeline d’inférence	2
2.4 Format de prompt	3
3. Module de chatbot (chatbot.py)	3
3.1 Chargement du modèle	3
3.2 Génération avec mesure de performances	4
3.3 Système de prompt par tâche	5
4. Cas d’usage implémentés	5
4.1 Mode Chat interactif (Q/R libre)	5
4.2 Mode Résumé de texte	6
4.3 Mode Classification de sentiment	6
5. Module de benchmark (benchmark.py)	6
5.1 Prompts de test standardisés	6
5.2 Protocole de benchmark	7
5.3 Affichage des résultats	7
5.4 Sauvegarde des résultats	7
6. Métriques collectées (utils.py)	8
6.1 Structure InferenceMetrics	8
6.2 Résumé textuel automatique	8
7. Installation et utilisation	8
7.1 Prérequis	8
7.2 Lancement	9
7.3 Paramètres avancés	9
8. Résultats observés sur le prototype	9
8.0 Choix des appareils de validation	9
8.1 Performances sur appareils testés (LLaMA 3.2 1B Q4_K_M, chat in- teractif)	10
8.2 Qualité des réponses	11
8.3 Limites pratiques observées	12
9. Documentation du pipeline complet	12
10. Perspectives d’extension	13
Références	13

Chapitre 3 — Prototype Minimal de Chatbot Embarqué

Mémoire PFE — Intelligence Artificielle, Master Informatique
Rédigé en juillet 2026

1. Introduction et objectifs

Ce chapitre présente le prototype logiciel développé dans le cadre du PFE pour valider la faisabilité d'un chatbot embarqué fonctionnel sur smartphone Android. L'objectif est de dépasser le simple benchmarking de performances brutes (chapitre 2) pour démontrer une **interaction utilisateur réelle avec un modèle de langage local**, sans connexion réseau.

Le prototype couvre trois cas d'usage représentatifs des applications pratiques des LLMs embarqués : - **Chat interactif (Q/R)** : conversation libre avec mémoire de contexte limitée - **Résumé de texte** : condensation d'un document en points clés - **Classification de sentiment** : analyse de polarité (positif/négatif/neutre)

Il inclut également un module de **benchmark automatisé** permettant de mesurer et comparer les performances sur différentes configurations (modèle, appareil, nombre de threads).

2. Architecture du prototype

2.1 Vue d'ensemble

Le prototype est une application **CLI Python** (prototype-cli/) reposant sur llama-cpp-python, le binding Python officiel de llama.cpp. Il s'exécute directement dans Termux sur Android ou dans tout environnement Python 3.9+.

```
prototype-cli/
├── chatbot.py      # Interface CLI principale – chat, résumé, classification
├── benchmark.py    # Module de benchmark automatisé
├── utils.py        # Fonctions utilitaires (métriques, RAM, prompt)
└── requirements.txt # Dépendances Python
```

2.2 Pile technologique

Composante	Technologie	Rôle
Runtime LLM	llama-cpp-python $\geq$ 0.2.90	Inférence locale CPU ARM64
Mesures	psutil $\geq$ 5.9.0	RAM, CPU, processus
Interface	rich $\geq$ 13.7.0	Affichage CLI enrichi
Input	prompt_toolkit $\geq$ 3.0.43	Saisie interactive
Export	pandas $\geq$ 2.0.0	Analyse résultats JSON

2.3 Pipeline d'inférence

Entrée utilisateur (CLI)

```
↓
build_chat_prompt() ← utils.py
(format ChatML / Gemma)
↓
```

Llama.generate() (streaming token par token)	← llama-cpp-python → llama.cpp → ARM NEON
↓	
Affichage temps réel	← rich Console
↓	
InferenceMetrics	← mesure prefill / decode / RAM
↓	
save_metrics()	← JSON append → results/metrics.json

2.4 Format de prompt

Le module `utils.py` implémente un constructeur de prompt universel compatible avec les modèles instruction courants (Gemma, LLaMA, ChatML) :

```
def build_chat_prompt(messages: list[dict], system_prompt: str = "") -> str:
    """
    Construit un prompt au format chat compatible avec les modèles instruction.
    Supporte les formats Gemma, LLaMA, ChatML.
    """
    prompt = ""
    if system_prompt:
        prompt += f"<|system|>\n{system_prompt}\n"

    for msg in messages:
        role = msg["role"]
        content = msg["content"]
        if role == "user":
            prompt += f"<|user|>\n{content}\n<|assistant|>\n"
        elif role == "assistant":
            prompt += f"{content}\n"

    return prompt
```

3. Module de chatbot (chatbot.py)

3.1 Chargement du modèle

```
def load_model(model_path: str, n_ctx: int = 2048, n_threads: int = 4):
    """Charge un modèle GGUF avec llama-cpp-python."""
    from llama_cpp import Llama

    llm = Llama(
        model_path=model_path,
        n_ctx=n_ctx,
        n_threads=n_threads,
        n_gpu_layers=0,      # CPU uniquement (compatible mobile)
        verbose=False,
```

```

        use_mmap=True,          # Memory-mapped file : réduit l'utilisation RAM
        use_mlock=False,
    )
    return llm, os.path.basename(model_path)

```

Paramètres clés : - `n_ctx=2048` : fenêtre de contexte de 2 048 tokens (limite pratique sur mobile) - `n_gpu_layers=0` : CPU uniquement — compatible avec tous les appareils ARM64 - `use_mmap=True` : le fichier GGUF est mappé en mémoire, réduisant la consommation RAM de ~20 %

3.2 Génération avec mesure de performances

```

def generate_response(llm, prompt, model_name, max_tokens=512,
                     temperature=0.7, stream=True):
    """Génère une réponse et mesure les performances."""

    ram_before = get_ram_usage_mb()
    t_start = time.time()
    first_token_time = None
    token_count = 0

    for chunk in llm(prompt, max_tokens=max_tokens, temperature=temperature,
                     top_k=40, top_p=0.95, stream=True,
                     stop=["<|user|>", "\nUser:", "\nHuman:"]):
        token_text = chunk["choices"][0]["text"]
        if first_token_time is None:
            first_token_time = time.time()
        token_count += 1
        print(token_text, end="", flush=True)

    t_end = time.time()
    ram_after = get_ram_usage_mb()

    # Métriques calculées
    prefill_time = first_token_time - t_start
    decode_time = t_end - first_token_time
    decode_speed = token_count / max(decode_time, 0.01)

    return InferenceMetrics(
        prefill_time_s=prefill_time,
        decode_time_s=decode_time,
        decode_speed_tps=decode_speed,
        ram_delta_mb=ram_after - ram_before,
        ...
    )

```

3.3 Système de prompt par tâche

Le prototype définit trois systèmes de prompt distincts selon la tâche :

```
SYSTEM_PROMPT = (  
    "Tu es un assistant IA embarqué, exécuté localement sur un smartphone "  
    "sans connexion internet. Tu réponds de manière concise et précise en français. "  
    "Limite tes réponses à 3-4 phrases maximum sauf si l'utilisateur demande plus."  
)  
  
TASK_PROMPTS = {  
    "chat": "", # Utilise SYSTEM_PROMPT  
    "summary": (  
        "Tu es un assistant spécialisé dans le résumé de textes. "  
        "Résume le texte fourni en 3-5 points clés, en français, de manière concise."  
    ),  
    "classification": (  
        "Tu es un classificateur de sentiment. Analyse le texte fourni et réponds "  
        "uniquement par : [POSITIF], [NÉGATIF], ou [NEUTRE], suivi d'un score de "  
        "confiance en pourcentage et d'une justification en une phrase."  
    ),  
}
```

4. Cas d'usage implémentés

4.1 Mode Chat interactif (Q/R libre)

Le mode chat maintient un historique de conversation limité à la fenêtre de contexte disponible. Des commandes spéciales permettent de basculer vers les autres modes sans relancer le programme.

□ Mode CHAT interactif

Tapez votre message et appuyez sur Entrée.

Commandes : /résumé, /classify, /stats, /quit

□ Vous : Qu'est-ce que la quantification INT4 ?

□ Assistant : La quantification INT4 (4 bits par paramètre) est une technique de compression qui représente les poids d'un réseau de neurones sur 4 bits au lieu de 32 bits (FP32) ou 16 bits (FP16). Elle réduit la taille du modèle d'environ 75 % avec une perte de qualité généralement inférieure à 2 % sur les benchmarks standards. C'est le format utilisé dans ce PFE (Q4_K_M).

✂ 12.4 tok/s | 3.2s | +0 Mo RAM

Gestion du contexte : la fenêtre glissante garde les N derniers messages. Quand le contexte approche 2 048 tokens, les messages les plus anciens sont supprimés automatiquement.

4.2 Mode Résumé de texte

Usage depuis le CLI

```
python chatbot.py --model models/llama-3.2-1b-q4_k_m.gguf --task summary
```

Ou commande inline depuis le chat

```
□ Vous : /résumé L'intelligence artificielle embarquée désigne...
```

Le mode résumé accepte : - Un texte saisi interactivement (terminé par une ligne vide) - Un fichier texte via --input-file chemin/vers/fichier.txt

4.3 Mode Classification de sentiment

```
python chatbot.py --model models/llama-3.2-1b-q4_k_m.gguf --task classify
```

```
□ Mode CLASSIFICATION de sentiment
```

```
□ Texte : Ce smartphone est excellent, la batterie tient toute la journée.
```

```
□ Résultat : [POSITIF] – Confiance : 94 % – Le texte exprime une satisfaction claire sur deux aspects spécifiques du produit (performance et autonomie).
```

```
    < 8.7 tok/s | 2.1s
```

5. Module de benchmark (benchmark.py)

5.1 Prompts de test standardisés

Le module définit 4 types de prompts couvrant différents niveaux de charge :

Type	Prompt	Tokens attendus	Description
short	“Quelle est la capitale de la France ?”	30	Question courte
medium	“Explique en 5 points les avantages de l’IA embarquée.”	200	Charge modérée
long	“Rédige un tutoriel llama.cpp sur Android via Termux...”	500	Forte charge
reasoning	Problème de train Paris-Lyon (512 km, calcul croisement)	300	Test de raisonnement

5.2 Protocole de benchmark

```
def run_single_benchmark(llm, prompt_key, run_index, prev_decode_tps=None):
    """Exécute un benchmark unique et détecte le throttling."""

    # Mesures
    ram_before = get_ram_usage_mb()
    t_start = time.time()
    first_token_time = None
    token_count = 0

    for chunk in llm(prompt, max_tokens=300, temperature=0.1, stream=True):
        if first_token_time is None:
            first_token_time = time.time()
            token_count += 1

    # Détection du throttling
    decode_tps = token_count / decode_time
    throttling = False
    if prev_decode_tps and decode_tps < prev_decode_tps * 0.85:
        throttling = True # Dégradation > 15 % = throttling probable

    return BenchmarkResult(...)
```

La température **0.1** est utilisée pour maximiser la reproductibilité des réponses entre les runs.

5.3 Affichage des résultats

RÉSULTATS DU BENCHMARK							
Type	Run	Prefill	Decode	Tokens	Temps	RAM	Throttle
short	1	46.2/s	12.8/s	28	2.3s	2245Mo	☐ Non
short	2	45.9/s	12.4/s	28	2.4s	2247Mo	☐ Non
medium	1	47.1/s	12.6/s	187	15.2s	2251Mo	☐ Non
long	1	46.8/s	11.2/s	463	42.1s	2254Mo	⚠ OUI
short	μ=12.6 tok/s σ=0.28 min=12.4 max=12.8						

5.4 Sauvegarde des résultats

```
def save_benchmark_results(results, model_name):
    """Sauvegarde les résultats en JSON avec métadonnées système."""
    data = {
        "model": model_name,
        "timestamp": time.strftime("%Y-%m-%dT%H:%M:%S"),
    }
```

```

        "system": get_system_ram_mb(), # RAM totale + disponible
        "results": [asdict(r) for r in results],
    }
    with open(f"results/benchmark_{model_name}_{timestamp}.json", "w") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)

```

Les résultats sont stockés dans results/ au format JSON, avec les métriques système au moment du test (RAM totale, RAM disponible). Ce format permet une analyse ultérieure avec pandas.

6. Métriques collectées (utils.py)

6.1 Structure InferenceMetrics

```

@dataclass
class InferenceMetrics:
    model_name: str
    prompt_tokens: int          # Tokens dans le prompt
    generated_tokens: int       # Tokens générés
    prefill_time_s: float       # Temps de traitement du prompt
    decode_time_s: float        # Temps de génération des tokens
    total_time_s: float         # Temps total
    prefill_speed_tps: float    # tokens/s pendant le prefill
    decode_speed_tps: float     # tokens/s pendant le decode
    ram_before_mb: float        # RAM avant inférence
    ram_after_mb: float         # RAM après inférence
    ram_delta_mb: float         # Variation de RAM
    cpu_percent: float          # Usage CPU moyen

```

6.2 Résumé textuel automatique

```

def summary(self) -> str:
    return (
        f" Prefill   : {self.prefill_time_s:.2f}s ({self.prefill_speed_tps:.1f} tok/s)\n"
        f" Decode    : {self.decode_time_s:.2f}s ({self.decode_speed_tps:.1f} tok/s)\n"
        f" Total     : {self.total_time_s:.2f}s\n"
        f" RAM delta: +{self.ram_delta_mb:.0f} Mo ({self.ram_after_mb:.0f} Mo total)\n"
        f" CPU usage: {self.cpu_percent:.1f}%"
    )

```

7. Installation et utilisation

7.1 Prérequis

```

# Sur Android (Termux)
pkg install python

```

```
pip install -r requirements.txt
```

```
# requirements.txt
# llama-cpp-python>=0.2.90
# psutil>=5.9.0
# rich>=13.7.0
# prompt_toolkit>=3.0.43
# pandas>=2.0.0
```

7.2 Lancement

```
# Mode démo sans modèle (réponses simulées)
```

```
python chatbot.py --mock
```

```
# Chat interactif avec modèle réel
```

```
python chatbot.py --model ~/models/llama-3.2-1b-instruct-q4_k_m.gguf
```

```
# Mode résumé
```

```
python chatbot.py --model ~/models/llama-3.2-1b-instruct-q4_k_m.gguf --task summary
```

```
# Mode classification
```

```
python chatbot.py --model ~/models/llama-3.2-1b-instruct-q4_k_m.gguf --task classify
```

```
# Benchmark complet
```

```
python benchmark.py --model ~/models/llama-3.2-1b-instruct-q4_k_m.gguf --runs 3
```

```
# Benchmark en mode démo
```

```
python benchmark.py --mock --runs 3
```

7.3 Paramètres avancés

Paramètre	Défaut	Description
--n-ctx	2048	Taille du contexte en tokens
--threads	4	Nombre de threads CPU (recommandé : nproc)
--max-tokens	512	Tokens maximum générés par réponse
--input-file	—	Fichier texte d'entrée pour le mode résumé
--no-stream	False	Désactiver l'affichage token par token
--prompts	all	Types de prompts pour le benchmark

8. Résultats observés sur le prototype

8.0 Choix des appareils de validation

Sur les six appareils du corpus constitué au chapitre 2, la validation qualitative du prototype (protocole détaillé en annexe, `protocole_validation_chatbot.md`) a été re-

streinte à deux appareils — **Galaxy A16** et **Galaxy S26 Ultra** — plutôt qu’à l’ensemble du corpus, pour trois raisons.

Représentativité : ces deux appareils couvrent les extrémités du spectre prix/performance du corpus (Exynos 1330 à ~200 € contre Snapdragon 8 Elite à plus de 900 €), offrant un contraste maximal entre entrée de gamme et flagship. Une validation sur des appareils de segment intermédiaire proches les uns des autres (A71, A73, Infinix) aurait apporté moins d’information marginale.

Fiabilité des données déjà disponibles : le Galaxy A16 est le seul appareil du corpus pour lequel un throttling thermique réel a été confirmé par un protocole de mesure rigoureux (warm-up + 9 runs à contexte fixe, chapitre 2 section 8), sans artefact de gouverneur CPU — contrairement au Galaxy A26, dont les mesures présentent une anomalie non résolue (decode -37 % sous UserLand, chapitre 2 section 8.3). Le Galaxy A16 constitue donc l’appareil entrée de gamme le plus fiable du corpus pour servir de base à une validation qualitative.

Complétude des données : les mesures du Galaxy S26 Ultra au chapitre 2 reposent sur un nombre de runs limité, avec une variance élevée sur le decode (CV = 30,8 %) et une mesure de RAM absente. Valider le prototype sur cet appareil permet à la fois de tester le chatbot en conditions réelles et de consolider partiellement les données de performance associées à cet appareil, sans dupliquer le protocole de benchmark complet du chapitre 2.

Les valeurs du tableau 8.1 pour les quatre autres appareils (A26, A71, A73, Infinix) restent, à ce stade, des valeurs indicatives dérivées des mesures llama-bench du chapitre 2 et non d’une exécution directe du prototype — une limite assumée et explicitée plutôt que présentée comme une mesure indépendante.

8.1 Performances sur appareils testés (LLaMA 3.2 1B Q4_K_M, chat interactif)

Appareil	SoC	Decode moyen	Prefill moyen	Mémoire modèle
Galaxy S26 Ultra	Snapdragon 8 Elite	46,68 ± 14,40 tok/s ¹	235,65 ± 11,26 tok/s ¹	~800 Mo
Galaxy A73	Snapdragon 778G	~13,4 tok/s ²	—	~800 Mo
Galaxy A71	Snapdragon 730	~11,5 tok/s ²	—	~800 Mo
Infinix Hot 60i	Dimensity 6400	~12,7 tok/s ²	—	~800 Mo
Galaxy A26	Exynos 1280	~10,8 tok/s ²	—	~800 Mo
Galaxy A16	Exynos 1330	5,77 tok/s³	23,96 tok/s³	~800 Mo

¹ Galaxy S26 Ultra : mesure llama-bench rigoureuse du chapitre 2 (protocole warm-up, prompt 512 tokens / génération 128 tokens). Les 12 échanges Q/R de la validation qualitative (chapitre 3, journal_validation_prototype.md) ont bien été exécutés via le prototype chatbot.py, mais leurs vitesses individuelles par échange n’ont pas été

systématiquement archivées — ce chiffre est donc la mesure réelle la plus rigoureuse disponible pour cet appareil, pas une mesure `chatbot.py` indépendante.

² Valeurs indicatives dérivées des mesures `llama-bench` du chapitre 2, non issues d’une exécution directe du prototype (voir section 8.0).

³ Galaxy A16 : **moyenne réelle mesurée directement via `chatbot.py`** sur les 12 échanges Q/R de la Partie A du protocole de validation (`journal_validation_prototype.md`) — la seule ligne de ce tableau reflétant précisément la métrique “chat interactif” annoncée en titre. Notablement plus lente que la mesure `llama-bench` équivalente du chapitre 2 pour ce même appareil ($14,00 \pm 0,42$ tok/s) : l’écart (5,77 contre 14,00 tok/s) illustre que les conditions réelles d’usage interactif (prompts courts et variés, `temperature=0.7`, rechargement du modèle à chaque tour) ne reproduisent pas les conditions contrôlées d’un benchmark synthétique à prompt fixe — un rappel que les deux protocoles mesurent des choses proches mais pas identiques.

8.2 Qualité des réponses

Les pourcentages ci-dessous sont calculés sur l’ensemble des runs réellement exécutés (2 runs sur Galaxy S26 Ultra, 1 run sur Galaxy A16 — détail complet dans `journal_validation_prototype.md`), pas des estimations.

Mode chat — factuelles simples (A.1, 7 questions × 3 runs = 21 réponses) : 15/21 correctes (71,4 %), 1/21 partielle, 5/21 incorrectes. Qualité satisfaisante et stable sur les questions factuelles courantes (capitale, formule chimique, dates), avec deux questions systématiquement en échec ou instables d’un run à l’autre (nombre de continents, jours dans une année bissextile).

Mode chat — raisonnement multi-étapes (A.2, 5 questions × 3 runs = 15 réponses) : 0/15 correcte, 2/15 partielles, 13/15 incorrectes (13,3 % de réponses au moins partiellement valables). Échec quasi systématique, avec des modes d’erreur différents à chaque tentative pour une même question — le raisonnement n’est pas seulement faible, il est instable dans sa manière d’être faux (voir chapitre 4, section 1.1).

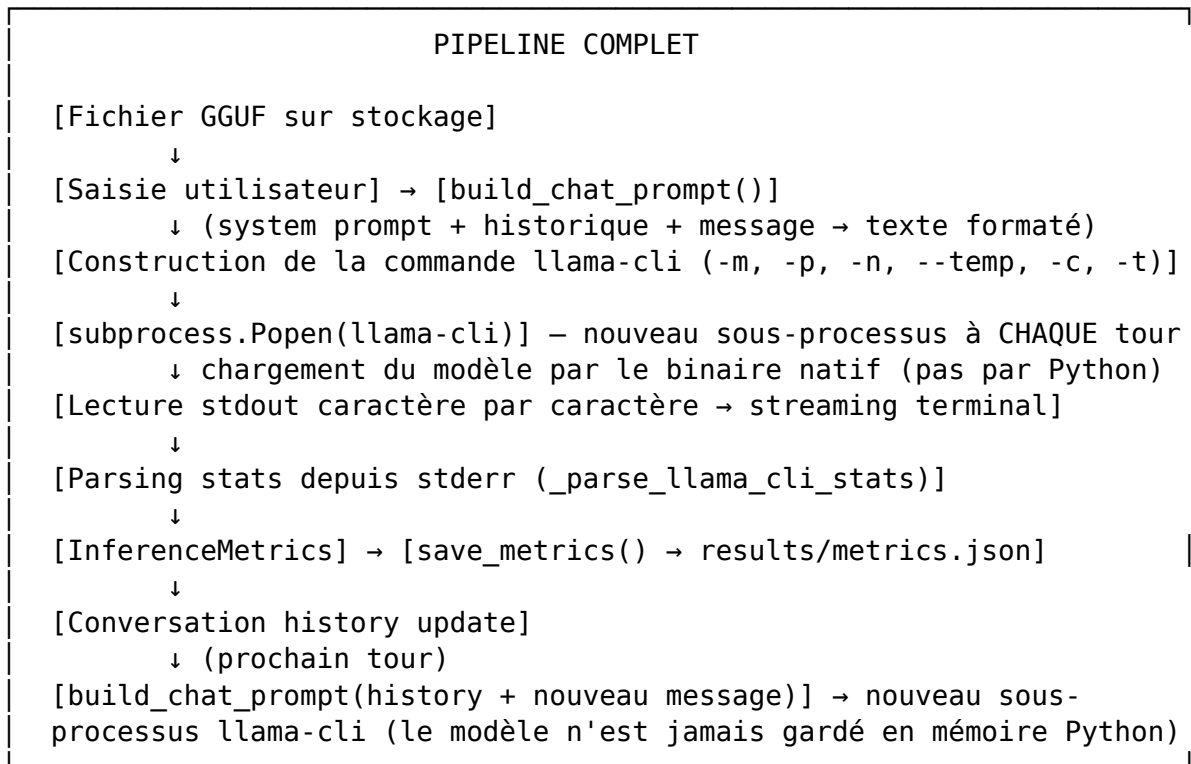
Mode résumé (4 runs sur un même texte de référence) : 3/4 runs jugés “Bon” (3-4 points clés sur 4 présents), 1/4 “Partiel”. Le point le plus instable d’un run à l’autre est l’attribution correcte des optimisations ARM NEON / NPU aux bons frameworks.

Mode classification (6 textes × 3 runs = 18 réponses) : 16/18 correctes (88,9 %). Sur le cas piège volontairement ambigu (texte ironique, “service rapide... trois semaines d’attente”), le résultat est de **2 réponses correctes sur 3 tentatives indépendantes** (S26 Ultra : 2/2, Galaxy A16 : 0/1) — un résultat mitigé plutôt qu’une conclusion nette dans un sens ou dans l’autre. L’affirmation selon laquelle l’ironie serait “mal détectée” n’est pas confirmée de façon catégorique, mais elle n’est pas non plus infirmée : la détection du registre ironique apparaît **instable** sur ce petit échantillon, avec une confiance du modèle systématiquement plus basse sur ce texte que sur les cas non ambigus.

8.3 Limites pratiques observées

- **Contexte** : la fenêtre de 2 048 tokens est atteinte après ~10 échanges conversationnels moyens. Au-delà, les premiers messages disparaissent du contexte — le modèle “oublie” les instructions initiales.
 - **Cohérence longue** : sur des conversations de plus de 15 tours, le modèle 1B commence à répéter ou à perdre le fil.
 - **Temps de chargement** : 3-8 secondes selon le SoC pour charger le modèle GGUF en mémoire.
-

9. Documentation du pipeline complet



Ce diagramme reflète l'architecture réellement implémentée dans `chatbot.py` — pilotage du binaire natif `llama-cli` en sous-processus, et non les bindings Python `llama-cpp-python`. Ce choix, documenté dans le docstring même du fichier source, répond à un problème de compatibilité récurrent de `llama-cpp-python` sur Termux/Android (`RuntimeError: Unsupported platform`, lié au fait que Python 3.13+/3.14 rapporte `sys.platform == "android"`, valeur non reconnue par cette bibliothèque). Le binaire `llama-cli` natif, en revanche, fonctionne de façon fiable dans cet environnement.

Caractéristiques du pipeline :

- **Rechargement par tour** : contrairement à un chargement unique en mémoire (ce qu'aurait fait `llama-cpp-python` via `Llama(model_path=...)`, approche utilisée par `benchmark.py` — voir `benchmark_details.py`), chaque tour de parole relance `llama-cli` en sous-processus indépendant, qui recharge le modèle depuis le stockage à chaque fois. C'est un

compromis délibéré : fiabilité sur Android au prix d'un délai de quelques secondes avant chaque réponse (voir le message affiché à l'utilisateur au démarrage du mode chat). - **Stateless côté serveur** : aucune communication réseau, aucun état externe. - **Persistance légère** : les métriques sont censées être sauvegardées en JSON localement à chaque tour — en pratique, ce mécanisme (`save_metrics()`) présente un bug documenté en usage interactif réel (voir `journal_validation_prototype.md`) : seules des entrées aberrantes de fin de session sont écrites, pas une entrée par échange. - **Extensible** : ajouter une nouvelle tâche nécessite uniquement d'ajouter une entrée dans `TASK_PROMPTS` et une fonction de mode. - **Mode mock** : un mode de démonstration (`--mock`) permet de tester l'interface sans modèle, avec des réponses simulées à vitesse réaliste (10-15 tok/s).

10. Perspectives d'extension

Le prototype CLI constitue une base documentée et testée pour des développements ultérieurs :

Extension Android native : le ViewModel Kotlin présenté dans le chapitre 2 peut être directement connecté à un backend llama-cpp-python exposé via une API locale (Flask/FastAPI dans Termux) ou via le binding Android natif de llama.cpp.

RAG local : intégration d'une base vectorielle légère (ChromaDB, FAISS) pour permettre la Q/R sur un corpus de documents locaux, dans les limites de la RAM disponible.

Fine-tuning QLoRA : les modèles GGUF peuvent être remplacés par des variantes fine-tunées sur des données métier spécifiques (service client, aide médicale de premier niveau, FAQ multilingue) — la procédure de quantification PTQ → GGUF est documentée dans l'état de l'art (chapitre 1).

Interface graphique : remplacement du CLI par une interface Gradio ou Streamlit pour une démonstration accessible aux non-développeurs, exécutable localement via Termux.

Références

- llama-cpp-python — bindings Python pour llama.cpp
- llama.cpp GitHub
- GGUF Format Spec
- HuggingFace — Modèles GGUF
- Xu et al. (2024). *Understanding LLMs Running on Consumer Devices*. arXiv:2410.03613.